“UNIVERSIDAD NACIONAL DE SAN AGUSTÍN”

FACULTAD DE INGENIERÍA PRODUCCIÓN Y SERVICIOS

ESCUELA PROFESIONAL DE CIENCIA DE LA COMPUTACIÓN



INFORME PRACTICA 5

TEMA: ANALIZADOR SINTÁCTICO SQL

CURSO: ESTRUCTURAS DE DATOS AVANZADOS

DOCENTE: MARCELA QUISPE CRUZ

ALUMNOS:

LUIS ENRIQUE RAMOS CHAMBI

ANGELA SHIRLETH SOTO HUERTA

TERCER AÑO - SEMESTRE B

AREQUIPA-PERÚ

08/11/26

1. Resumen y estado del proyecto

Este documento reúne lo desarrollado hasta ahora en la Práctica 5 (analizador sintáctico predictivo LL(1) no recursivo para sentencias SELECT). El trabajo se divide en dos partes: la **Parte A** (gramática y motor de análisis) ya está implementada, probada e integrada con el analizador léxico de la Práctica 4; la **Parte B** (tabla de parseo en código, traza, pruebas de entradas inválidas y ejecutable).

1.1 Estado por entregable

Entregable	Responsable	Estado	Comentario
1. Formalización de la gramática	A	**Hecho**	EBNF, BNF LL(1), FIRST/FOLLOW y verificación LL(1) (secciones 3 a 5).
2. Tabla de parseo M[A, a]	B	**Hecho**	Tabla derivada y verificada contra FIRST/FOLLOW.
3. Motor LL(1) por pila	A	**Hecho**	parser.cpp con pila explícita e integración con el lexer.
4. Manejo de errores	B	**Hecho**	Línea, columna, token encontrado y esperados.
5. Batería de pruebas	A (válidas, motor) · B (inválidas)	**Hecho**	36 pruebas pasan.
Traza de la pila	A (mecanismo) · B (impresión)	**Hecho**	El motor emite un evento por paso.
6. Informe PDF	A y B	**Hecho**	

2. Alcance del subconjunto SQL y decisiones de diseño

El equipo soporta una única sentencia SELECT por entrada, con la estructura mínima exigida por el enunciado:

```
Query ::= SELECT ( * | id { , id } ) FROM id [ WHERE Cond ] ;
Cond  ::= AndCond { OR AndCond }
AndCond ::= Pred { AND Pred }
Pred  ::= id RelOp Value
RelOp ::= = | <> | < | > | <= | >=
Value ::= int | float | string | id
```

Decisión	Detalle
Precedencia lógica	AND se agrupa antes que OR (por eso la condición tiene dos niveles: Cond y AndCond). No hay paréntesis porque el lexer no los reconoce.
Una sentencia por entrada	Después del “;” solo se acepta el fin de entrada. Una segunda sentencia se rechaza.
Value → id	Se permite comparar una columna con otra (a = b).
Modo “todo o nada”	Un error léxico rechaza la consulta sin llegar al parser; un error sintáctico detiene el análisis en el primer fallo y lo reporta. Las advertencias del lexer no bloquean.
Operador distinto	Tanto <> como != producen el mismo token y se tratan como el mismo terminal.
Palabras reservadas	SELECT, FROM, WHERE, AND y OR no pueden usarse como identificadores (el lexer las reconoce sin distinguir mayúsculas).

3. Formalización de la gramática

3.1 Terminales y correspondencia con el lexer

Terminal de la gramática	TokenType del lexer (Práctica 4)
SELECT, FROM, WHERE, AND, OR	KEYWORD_SELECT, KEYWORD_FROM, KEYWORD_WHERE, KEYWORD_AND, KEYWORD_OR
id	IDENTIFIER
int, float, string	INTEGER_LITERAL, FLOAT_LITERAL, STRING_LITERAL
=, <>, <, >, <=, >=	EQUAL, NOT_EQUAL, LESS_THAN, GREATER_THAN, LESS_EQUAL, GREATER_EQUAL
"," y ";"	COMMA, SEMICOLON
*	STAR (token nuevo agregado al lexer)
$	TOKEN_EOF

3.2 Gramática en BNF LL(1)

Las repeticiones `{ … }` de la versión EBNF se convierten en no terminales de cola (`IdListP`, `CondP`, `AndP`) recursivos por la derecha, con alternativa ε. Símbolo inicial: **Query**.

N.º	Producción
1	Query → SELECT SelectList FROM id WhereOpt ;
2	SelectList → *
3	SelectList → id IdListP
4	IdListP → , id IdListP
5	IdListP → ε
6	WhereOpt → WHERE Cond
7	WhereOpt → ε
8	Cond → AndCond CondP
9	CondP → OR AndCond CondP
10	CondP → ε
11	AndCond → Pred AndP
12	AndP → AND Pred AndP
13	AndP → ε
14	Pred → id RelOp Value
15	RelOp → =
16	RelOp → <>
17	RelOp → <
18	RelOp → >
19	RelOp → <=
20	RelOp → >=
21	Value → int
22	Value → float
23	Value → string
24	Value → id

3.3 Eliminación de recursión a la izquierda y factorización

La forma natural de escribir las condiciones y las listas es recursiva por la izquierda, lo que haría que un parser predictivo entre en un bucle infinito:

```
Cond   → Cond OR AndCond | AndCond          (recursiva por la izquierda)
AndCond→ AndCond AND Pred | Pred            (recursiva por la izquierda)
SelectList → SelectList , id | id           (recursiva por la izquierda)
```

Se aplica la transformación estándar **A → Aα | β ⇒ A → βA′, A′ → αA′ | ε**:

```
Cond   → AndCond CondP        CondP → OR AndCond CondP | ε
AndCond→ Pred AndP            AndP  → AND Pred AndP    | ε
SelectList → id IdListP       IdListP → , id IdListP   | ε
```

Factorización: ningún no terminal tiene dos alternativas con prefijo común. `SelectList` se decide con el primer token (`*` o `id`), y `Value` y `RelOp` empiezan siempre con un terminal distinto en cada alternativa. Por eso no hace falta factorizar nada adicional. La gramática tampoco tiene recursión izquierda indirecta, porque todo ciclo pasa por la derecha de una cola.

4. Conjuntos FIRST y FOLLOW

4.1 Definiciones y cálculo

FIRST(α) es el conjunto de terminales con los que puede empezar una cadena derivada de α (más ε si α puede desaparecer). FOLLOW(A) es el conjunto de terminales que pueden aparecer inmediatamente después de A en alguna forma sentencial; contiene `$` si A es el símbolo inicial.

- **Anulables** (derivan ε): IdListP, WhereOpt, CondP, AndP.
- **FIRST** de un terminal es él mismo. Para A → $X_1X_2\ldots$, se toma FIRST(X_1) y, mientras X_i sea anulable, también FIRST(X_{i+1}); si toda la cadena es anulable, se agrega ε.
- **FOLLOW** de B en A → αBβ: FIRST(β) sin ε; y si β es anulable (o vacía), también FOLLOW(A).

El código calcula ambos por punto fijo (se repite hasta que ningún conjunto cambia) y las pruebas verifican que coinciden con esta tabla.

4.2 Resultado

No terminal	FIRST	FOLLOW	Justificación del FOLLOW
Query	{ SELECT }	{ $ }	Símbolo inicial.
SelectList	{ *, id }	{ FROM }	Aparece antes de FROM (prod. 1).
IdListP	{ ",", ε }	{ FROM }	Está al final de SelectList (prod. 3): hereda FOLLOW(SelectList).
WhereOpt	{ WHERE, ε }	{ ; }	Aparece antes de ";" (prod. 1).
Cond	{ id }	{ ; }	Está al final de WhereOpt (prod. 6): hereda FOLLOW(WhereOpt).
CondP	{ OR, ε }	{ ; }	Está al final de Cond (prod. 8): hereda FOLLOW(Cond).
AndCond	{ id }	{ OR, ; }	Le sigue CondP: FIRST(CondP)−ε = {OR}; como CondP es anulable, también FOLLOW(Cond)/FOLLOW(CondP) = {;}.
AndP	{ AND, ε }	{ OR, ; }	Está al final de AndCond (prod. 11): hereda FOLLOW(AndCond).
Pred	{ id }	{ AND, OR, ; }	Le sigue AndP: FIRST(AndP)−ε = {AND}; AndP es anulable, hereda FOLLOW(AndCond) = {OR, ;}.

No terminal	FIRST	FOLLOW	Justificación del FOLLOW
RelOp	{ =, <>, <, >, <=, >= }	{ id, int, float, string }	Le sigue Value (prod. 14): FIRST(Value).
Value	{ int, float, string, id }	{ AND, OR, ; }	Está al final de Pred (prod. 14): hereda FOLLOW(Pred).

5. Verificación de la condición LL(1)

Para cada producción A → α, el conjunto de selección es SEL(A → α) = FIRST(α), más FOLLOW(A) si α es anulable. Solo los no terminales con varias alternativas pueden tener conflicto; sus conjuntos deben ser disjuntos:

No terminal	Prods.	Conjuntos de selección	¿Disjuntos?
SelectList	2 / 3	{ * } / { id }	Sí
IdListP	4 / 5	{ "," } / FOLLOW = { FROM }	Sí
WhereOpt	6 / 7	{ WHERE } / FOLLOW = { ; }	Sí
CondP	9 / 10	{ OR } / FOLLOW = { ; }	Sí
AndP	12 / 13	{ AND } / FOLLOW = { OR, ; }	Sí
RelOp	15 a 20	Un operador distinto por producción	Sí
Value	21 a 24	{ int }, { float }, { string }, { id }	Sí

Conclusión: no hay conflictos; la gramática es LL(1). Cada una de las 25 celdas ocupadas de la tabla pertenece a una sola producción (sección 6). Esta comprobación también está automatizada en las pruebas: para cada producción, su conjunto de selección coincide exactamente con las celdas que ocupa en la tabla de referencia.

6. Tabla de parseo LL(1) M[A, a]

Cada celda contiene el número de la producción (sección 3.2) que se debe aplicar cuando el no terminal A está en el tope de la pila y el siguiente token es a. **Celda vacía = error sintáctico.** Regla de llenado: para cada A → α, se escribe en M[A, a] para todo a en FIRST(α) y, si α es anulable, también para todo a en FOLLOW(A).

A \ a	SELECT	FROM	WHERE	AND	OR	id	int	float	string	=	<>	<	>	<=	>=	,	;	*	$
Query	1																		
SelectList						3												2	
IdListP		5														4			
WhereOpt			6														7		
Cond						8													
CondP					9												10		
AndCond						11													
AndP				12	13												13		
Pred						14													
RelOp										15	16	17	18	19	20				
Value						24	21	22	23										

La tabla tiene **25 celdas ocupadas** y ningún conflicto. Las producciones con ε se ubican por FOLLOW: por ejemplo, `AndP → ε` (13) ocupa M[AndP, OR] y M[AndP, ;] porque FOLLOW(AndP) = { OR, ; }.

Representación en código: un mapa `std::map<std::pair<NonTerminal, Terminal>, int>` (estructura `ParseTable`, en `parse_table.h`) que asocia cada par con el índice de la producción; `lookup()` devuelve −1 si la celda está vacía. La construcción automática de esta tabla a partir de FIRST/FOLLOW. Mientras tanto, las pruebas del motor usan una copia de esta misma tabla escrita a mano, de modo que el motor ya se puede verificar sin esperar a la parte B.

Cómo compilar:

```
None
g++ -std=c++20 -Iinclude -Iparser_ll1/include src/lexer.cpp src/token.cpp src/input.cpp parser_ll1/src/grammar.cpp
parser_ll1/src/parser.cpp parser_ll1/src/parse_table.cpp parser_ll1/src/trace.cpp parser_ll1/src/main.cpp -o parser_ll1.exe
```

Cada archivo hace lo siguiente:

- `grammar.cpp` define las 24 producciones y calcula FIRST y FOLLOW.
- `parse_table.cpp` construye la tabla M[A,a].
- `parser.cpp` es el algoritmo con pila.
- `trace.cpp` imprime la traza.

2. Ver la tabla

siguiente paso :

```
None
.\parser_ll1.exe --tables
```

```
PS C:\Users\HP SUPPORT\Downloads\EDA-Lab04-main2> g++ -std=c++20 -Iinclude -Iparser_ll1/include src/lexer.cpp src/token.cpp src/input.cpp pa
r_ll1/src/grammar.cpp parser_ll1/src/parser.cpp parser_ll1/src/parse_table.cpp parser_ll1/src/trace.cpp parser_ll1/src/main.cpp -o parser_ll
xe
PS C:\Users\HP SUPPORT\Downloads\EDA-Lab04-main2> .\parser_ll1.exe --tables
Gramatica LL(1) (simbolo inicial: Query)
   1. Query -> SELECT SelectList FROM id WhereOpt ;
   2. SelectList -> *
   3. SelectList -> id IdListP
   4. IdListP -> , id IdListP
   5. IdListP -> eps
   6. WhereOpt -> WHERE Cond
   7. WhereOpt -> eps
   8. Cond -> AndCond CondP
   9. CondP -> OR AndCond CondP
  10. CondP -> eps
  11. AndCond -> Pred AndP
  12. AndP -> AND Pred AndP
  13. AndP -> eps
  14. Pred -> id RelOp Value
  15. RelOp -> =
  16. RelOp -> <>
  17. RelOp -> <
  18. RelOp -> >
  19. RelOp -> <=
  20. RelOp -> >=
  21. Value -> int
  22. Value -> float
  23. Value -> string
  24. Value -> id
```

```
Conjuntos FIRST
  FIRST(Query) = { SELECT }
  FIRST(SelectList) = { id, * }
  FIRST(IdListP) = { ',', eps }
  FIRST(WhereOpt) = { WHERE, eps }
  FIRST(Cond) = { id }
  FIRST(CondP) = { OR, eps }
  FIRST(AndCond) = { id }
  FIRST(AndP) = { AND, eps }
  FIRST(Pred) = { id }
  FIRST(RelOp) = { =, <>, <, >, <=, >= }
  FIRST(Value) = { id, int, float, string }
Conjuntos FOLLOW
  FOLLOW(Query) = { $ }
  FOLLOW(SelectList) = { FROM }
  FOLLOW(IdListP) = { FROM }
  FOLLOW(WhereOpt) = { ; }
  FOLLOW(Cond) = { ; }
  FOLLOW(CondP) = { ; }
  FOLLOW(AndCond) = { OR, ; }
  FOLLOW(AndP) = { OR, ; }
  FOLLOW(Pred) = { AND, OR, ; }
  FOLLOW(RelOp) = { id, int, float, string }
  FOLLOW(Value) = { AND, OR, ; }
```

```
Tabla de parseo LL(1) M[A, a]   (celda = N de produccion; vacia = error)
           SELECT FROM   WHERE  AND    OR     id     int    float  string =     <>    <     >     <=    >=    ,     ;     *     $

-------------------------------------------------------------------------------------------------------------------------------------
--
Query      1      .      .      .      .      .      .      .      .      .      .      .      .      .      .      .      .      .
SelectList .      .      .      .      .      3      .      .      .      .      .      .      .      .      .      .      2      .
IdListP    .      5      .      .      .      .      .      .      .      .      .      .      .      .      .      4      .      .
WhereOpt   .      .      6      .      .      .      .      .      .      .      .      .      .      .      .      .      7      .
Cond       .      .      .      .      .      8      .      .      .      .      .      .      .      .      .      .      .      .
CondP      .      .      .      .      9      .      .      .      .      .      .      .      .      .      .      .      10     .
AndCond    .      .      .      .      .      11     .      .      .      .      .      .      .      .      .      .      .      .
AndP       .      .      .      12     13     .      .      .      .      .      .      .      .      .      .      .      13     .
Pred       .      .      .      .      .      14     .      .      .      .      .      .      .      .      .      .      .      .
RelOp      .      .      .      .      .      .      .      .      .      15     16     17     18     19     20     .      .      .
Value      .      .      .      .      .      24     21     22     23     .      .      .      .      .      .      .      .      .
```

Muestra la gramática, FIRST/FOLLOW y la tabla M[A,a] con sus 25 celdas. Termina con “Sin conflictos: la gramatica es LL(1)”. Haz una captura de esa pantalla para el informe.

Ver el algoritmo (la pila paso a paso)

```
None
.\parser_ll1.exe parser_ll1\examples\parser_valid_simple.sql
```

```
Sin conflictos: la gramatica es LL(1).
PS C:\Users\HP SUPPORT\Downloads\EDA-Lab04-main2> .\parser_ll1.exe parser_ll1\examples\parser_valid_simple.sql
Consulta:
SELECT * FROM estudiantes;


Traza del parseo:
Paso  Pila (tope a la izq.)                    Entrada                          Accion
-------------------------------------------------------------------------------------------------------------
1     Query $                                  SELECT * FROM estudiantes ; $    Query -> SELECT SelectList FROM id WhereOpt ;
2     SELECT SelectList FROM id WhereOpt ; $   SELECT * FROM estudiantes ; $    match SELECT
3     SelectList FROM id WhereOpt ; $          * FROM estudiantes ; $           SelectList -> *
4     * FROM id WhereOpt ; $                   * FROM estudiantes ; $           match *
5     FROM id WhereOpt ; $                     FROM estudiantes ; $             match FROM
6     id WhereOpt ; $                          estudiantes ; $                  match id
7     WhereOpt ; $                             ; $                              WhereOpt -> eps
8     ; $                                      ; $                              match ;
9     $                                        $                                ACEPTAR

RESULTADO: ACEPTADA (sintacticamente valida)
Producciones aplicadas: 1 2 7
PS C:\Users\HP SUPPORT\Downloads\EDA-Lab04-main2>
```

7. Motor de análisis LL(1) basado en pila

7.1 Algoritmo

El motor (`parseTokens`, en `parser.cpp`) es iterativo, usa una pila explícita y no tiene recursión:

```
pila ← [ $ , Query ]                    (el tope es el último elemento)
pos  ← 0
repetir:
    a ← terminal del token pos            (si no hay más tokens: $)
    X ← tope de la pila
    si X es terminal:
        si X = a y a = $ :  ACEPTAR
        si X = a         :  quitar X de la pila; pos ← pos + 1      (match)
        si no            :  ERROR (se esperaba X)
    si no (X es no terminal):
        p ← M[X, a]
        si la celda está vacía:  ERROR (se esperaba FIRST/FOLLOW de X)
        si no: quitar X; apilar el lado derecho de p en orden inverso   (expansión)
```

7.2 Diseño en C++

Elemento	Función
`Symbol`, `Production`	Representan terminales, no terminales y las 24 producciones (`grammar.h`).
`ParseTable`	Mapa (A, a) → producción, con `lookup()` y `expected()` (`parse_table.h`).
`parseTokens(tokens, table, observer)`	Ejecuta el algoritmo. Devuelve `ParseResult` con `accepted`, el `error` (si lo hay) y la lista de producciones aplicadas.
`StepObserver` / `StepEvent`	Gancho opcional: el motor lo invoca en cada paso con la pila, la posición en la entrada y la acción. Permite construir la traza sin modificar el bucle.
`terminalFromToken`	Convierte el `TokenType` del lexer en el terminal de la gramática.

7.3 Integración con el analizador léxico

El parser consume directamente `std::vector<Token>` generado por `Lexer::tokenizeAll()` y no modifica el código del lexer, salvo la incorporación del token `STAR`, necesaria para `SELECT *` (antes `*` era un error léxico). Si el vector llega sin `TOKEN_EOF` o vacío, el motor usa un fin de entrada sintético en la última posición conocida, así que nunca accede fuera de rango.

Reglas de integración: un error léxico rechaza la consulta antes de analizarla; las advertencias (por ejemplo, `.5`) no la bloquean.

8. Manejo de errores sintácticos

8.1 Ejemplos reales

Entrada	Mensaje del parser
`FROM t SELECT a;`	Error sintactico en 1:1: se encontro 'FROM' pero se esperaba SELECT
`SELECT a FROM ;`	Error sintactico en 1:15: se encontro ';' pero se esperaba identificador
`SELECT a, FROM t;`	Error sintactico en 1:11: se encontro 'FROM' pero se esperaba identificador
`SELECT a FROM t WHERE x 1;`	Error sintactico en 1:25: se encontro '1' pero se esperaba '=', '<>', '<', '>', '<=' o '>='
`SELECT a FROM t WHERE x = 1`	Error sintactico en 1:28: se encontro fin de entrada pero se esperaba AND, OR o ';'
`SELECT a FROM t WHERE x = 1 AND ;`	Error sintactico en 1:33: se encontro ';' pero se esperaba identificador

- **.Consultas válidas simples: Verificación de sentencias con la estructura mínima definida por el grupo**

VÁLIDOS:

```
PS C:\Users\HP SUPPORT\Downloads\EDA-Lab04-main2> .\parser_ll1.exe --quiet parser_ll1\examples\parser_valid_simple.sql
Consulta:
SELECT * FROM estudiantes;


RESULTADO: ACEPTADA (sintacticamente valida)
Producciones aplicadas: 1 2 7
PS C:\Users\HP SUPPORT\Downloads\EDA-Lab04-main2>
```

Figura: Ejecución exitosa de la consulta válida simple `SELECT * FROM estudiantes;`, aceptada correctamente por el analizador sintáctico tras aplicar las producciones 1, 2 y 7.

- **Consultas válidas complejas: Proyecciones múltiples, combinaciones de cláusulas op cionales (WHERE) y expresiones compuestas.**

```
PS C:\Users\HP SUPPORT\Downloads\EDA-Lab04-main2> .\parser_ll1.exe --quiet parser_ll1\examples\parser_valid_complex.sql
Consulta:
SELECT nombre, edad, carrera
FROM estudiantes
WHERE edad >= 18 AND carrera = 'CC' OR promedio <> 10.5 AND codigo != 7;


RESULTADO: ACEPTADA (sintacticamente valida)
Producciones aplicadas: 1 3 4 4 5 6 8 11 14 20 21 12 14 15 23 13 9 11 14 16 22 12 14 16 21 13 10
PS C:\Users\HP SUPPORT\Downloads\EDA-Lab04-main2>
```

- **Detección de errores sintácticos: Pruebas con palabras clave fuera de orden, omisión de tokens obligatorios (como el nombre de la tabla o el delimitador final) y expresiones mal formadas**

ERRORES:

Para comprobar el manejo de errores se diseñó una batería de seis consultas inválidas, una por cada categoría exigida en el enunciado. En todos los casos el analizador rechazó la entrada, informó la posición exacta (línea:columna), el token encontrado y los tokens esperados, y terminó sin colapsar.

```
None
.\parser_ll1.exe --quiet parser_ll1\examples\parser_invalid.sql
.\parser_ll1.exe --quiet parser_ll1\examples\parser_invalid_sin_tabla.sql
.\parser_ll1.exe --quiet parser_ll1\examples\parser_invalid_sin_puntoycoma.sql
.\parser_ll1.exe --quiet parser_ll1\examples\parser_invalid_coma.sql
.\parser_ll1.exe --quiet parser_ll1\examples\parser_invalid_expresion.sql
.\parser_ll1.exe --quiet parser_ll1\examples\parser_invalid_lexico.sql
```

- En `parser_invalid.sql` las palabras clave están en desorden o fuera de orden (`FROM estudiantes SELECT nombre;`). Con `Query` en el tope de la pila, el parser solo acepta `SELECT` como primer token. Encontró `FROM` en la posición 1:1, así que la celda M[Query, FROM] está vacía y se reporta el error.

```
PS C:\Users\HP SUPPORT\Downloads\EDA-Lab04-main2> .\parser_ll1.exe --quiet parser_ll1\examples\parser_invalid.sql
Consulta:
FROM estudiantes SELECT nombre;


Error sintactico en 1:1: se encontro 'FROM' pero se esperaba SELECT
RESULTADO: RECHAZADA (error sintactico)
```

- En `parser_invalid_sin_tabla.sql` falta el nombre de la tabla (`SELECT nombre, edad FROM ;`). Tras `FROM` el parser espera un identificador, pero encontró `;` en 1:26. Es un *mismatch* entre el terminal `id` del tope de la pila y el token actual.

```
RESULTADO: RECHAZADA (error sintactico)
PS C:\Users\HP SUPPORT\Downloads\EDA-Lab04-main2> .\parser_ll1.exe --quiet parser_ll1\examples\parser_invalid_sin_tabla.sql
Consulta:
SELECT nombre, edad FROM ;


Error sintactico en 1:26: se encontro ';' pero se esperaba identificador
RESULTADO: RECHAZADA (error sintactico)
```

- En `parser_invalid_sin_puntoycoma.sql` se omitió el delimitador final. La entrada termina después de `estudiantes`, y en 2:1 el parser encontró el fin de entrada cuando esperaba `WHERE` o `;`. Esos dos tokens salen de la fila de `WhereOpt` en la tabla.

```
PS C:\Users\HP SUPPORT\Downloads\EDA-Lab04-main2> .\parser_ll1.exe --quiet parser_ll1\examples\parser_invalid_sin_puntoycoma.sql
>>
Consulta:
SELECT nombre FROM estudiantes


Error sintactico en 2:1: se encontro fin de entrada pero se esperaba WHERE o ';'
RESULTADO: RECHAZADA (error sintactico)
PS C:\Users\HP SUPPORT\Downloads\EDA-Lab04-main2>
```

- En `parser_invalid_coma.sql` hay una coma sobrante (`SELECT nombre, FROM estudiantes;`). Después de la coma, la producción `IdListP → , id IdListP` exige otro identificador, pero apareció `FROM` en 1:16.

```
luisenramos@ElAprendiz2005:~/Documentos/EDA/EDA-Lab04/build$ ./parser_ll1 ../parser_ll1/examples/parser_invalid_coma.sql
Consulta:
SELECT nombre, FROM estudiantes;


Traza del parseo:
Paso  Pila (tope a la izq.)                    Entrada                              Accion
-------------------------------------------------------------------------------------------------------------------
1     Query $                                  SELECT nombre , FROM estudiantes ; $     Query -> SELECT SelectList FROM id WhereOpt ;
2     SELECT SelectList FROM id WhereOpt ; $   SELECT nombre , FROM estudiantes ; $     match SELECT
3     SelectList FROM id WhereOpt ; $          nombre , FROM estudiantes ; $            SelectList -> id IdListP
4     id IdListP FROM id WhereOpt ; $          nombre , FROM estudiantes ; $            match id
5     IdListP FROM id WhereOpt ; $             , FROM estudiantes ; $                   IdListP -> , id IdListP
6     , id IdListP FROM id WhereOpt ; $        , FROM estudiantes ; $                   match ,
7     id IdListP FROM id WhereOpt ; $          FROM estudiantes ; $                     ERROR: se esperaba identificador

Error sintactico en 1:16: se encontro 'FROM' pero se esperaba identificador
RESULTADO: RECHAZADA (error sintactico)
```

- En `parser_invalid_expresion.sql` la expresión está mal formada: falta el valor después del operador (`WHERE edad >= ;`). Tras >= el parser espera un `Value`, es decir un identificador, entero, decimal o cadena, y encontró `;` en 1:46. En este caso la celda M[Value, ;] está vacía.

```
PS C:\Users\HP SUPPORT\Downloads\EDA-Lab04-main2> .\parser_ll1.exe --quiet parser_ll1\examples\parser_invalid_expresion.sql
>>
Consulta:
SELECT nombre FROM estudiantes WHERE edad >= ;


Error sintactico en 1:46: se encontro ';' pero se esperaba identificador, entero, decimal o cadena
RESULTADO: RECHAZADA (error sintactico)
PS C:\Users\HP SUPPORT\Downloads\EDA-Lab04-main2>
```

- En `parser_invalid_lexico.sql` el error es léxico: el carácter `@` no pertenece al lenguaje. Lo detecta el analizador léxico en 1:52, antes del análisis sintáctico, y la consulta se rechaza como `RECHAZADA (errores lexicos)` sin llegar al parser

```
PS C:\Users\HP SUPPORT\Downloads\EDA-Lab04-main2> .\parser_ll1.exe --quiet parser_ll1\examples\parser_invalid_lexico.sql
Consulta:
SELECT nombre FROM estudiantes WHERE edad > 18 AND @ carrera = 1;

[ERROR lexico] 1:52 Character not valid '@'

RESULTADO: RECHAZADA (errores lexicos)
PS C:\Users\HP SUPPORT\Downloads\EDA-Lab04-main2>
```

9. Traza de la pila sintáctica

Consulta analizada (válida): `SELECT nombre, edad FROM alumnos WHERE edad >= 18 AND ciclo = 3;`. Cada fila es el estado antes de aplicar la acción; la pila se lee con el **tope a la izquierda** y `$` marca el fondo. Se generó con el motor real y su observador de pasos. Producciones aplicadas, en orden: 1, 3, 4, 5, 6, 8, 11, 14, 20, 21, 12, 14, 15, 21, 13, 10 (32 pasos: 16 expansiones, 15 emparejamientos y la aceptación).

Paso	Pila	Entrada restante	Acción
1	`Query $`	`SELECT nombre , edad FROM alumnos WHERE edad >= 18 AND ciclo = 3 ; $`	`Query -> SELECT SelectList FROM id WhereOpt ;`
2	`SELECT SelectList FROM id WhereOpt ; $`	`SELECT nombre , edad FROM alumnos WHERE edad >= 18 AND ciclo = 3 ; $`	`match SELECT`
3	`SelectList FROM id WhereOpt ; $`	`nombre , edad FROM alumnos WHERE edad >= 18 AND ciclo = 3 ; $`	`SelectList -> id IdListP`
4	`id IdListP FROM id WhereOpt ; $`	`nombre , edad FROM alumnos WHERE edad >= 18 AND ciclo = 3 ; $`	`match id`
5	`IdListP FROM id WhereOpt ; $`	`, edad FROM alumnos WHERE edad >= 18 AND ciclo = 3 ; $`	`IdListP -> , id IdListP`
6	`, id IdListP FROM id WhereOpt ; $`	`, edad FROM alumnos WHERE edad >= 18 AND ciclo = 3 ; $`	`match ,`
7	`id IdListP FROM id WhereOpt ; $`	`edad FROM alumnos WHERE edad >= 18 AND ciclo = 3 ; $`	`match id`
8	`IdListP FROM id WhereOpt ; $`	`FROM alumnos WHERE edad >= 18 AND ciclo = 3 ; $`	`IdListP -> eps`
9	`FROM id WhereOpt ; $`	`FROM alumnos WHERE edad >= 18 AND ciclo = 3 ; $`	`match FROM`
10	`id WhereOpt ; $`	`alumnos WHERE edad >= 18 AND ciclo = 3 ; $`	`match id`
11	`WhereOpt ; $`	`WHERE edad >= 18 AND ciclo = 3 ; $`	`WhereOpt -> WHERE Cond`
12	`WHERE Cond ; $`	`WHERE edad >= 18 AND ciclo = 3 ; $`	`match WHERE`
13	`Cond ; $`	`edad >= 18 AND ciclo = 3 ; $`	`Cond -> AndCond CondP`
14	`AndCond CondP ; $`	`edad >= 18 AND ciclo = 3 ; $`	`AndCond -> Pred AndP`
15	`Pred AndP CondP ; $`	`edad >= 18 AND ciclo = 3 ; $`	`Pred -> id RelOp Value`
16	`id RelOp Value AndP CondP ; $`	`edad >= 18 AND ciclo = 3 ; $`	`match id`
17	`RelOp Value AndP CondP ; $`	`>= 18 AND ciclo = 3 ; $`	`RelOp -> >=`
18	`>= Value AndP CondP ; $`	`>= 18 AND ciclo = 3 ; $`	`match >=`
19	`Value AndP CondP ; $`	`18 AND ciclo = 3 ; $`	`Value -> int`
20	`int AndP CondP ; $`	`18 AND ciclo = 3 ; $`	`match int`
21	`AndP CondP ; $`	`AND ciclo = 3 ; $`	`AndP -> AND Pred AndP`
22	`AND Pred AndP CondP ; $`	`AND ciclo = 3 ; $`	`match AND`
23	`Pred AndP CondP ; $`	`ciclo = 3 ; $`	`Pred -> id RelOp Value`
24	`id RelOp Value AndP CondP ; $`	`ciclo = 3 ; $`	`match id`
25	`RelOp Value AndP CondP ; $`	`= 3 ; $`	`RelOp -> =`
26	`= Value AndP CondP ; $`	`= 3 ; $`	`match =`
27	`Value AndP CondP ; $`	`3 ; $`	`Value -> int`
28	`int AndP CondP ; $`	`3 ; $`	`match int`
29	`AndP CondP ; $`	`; $`	`AndP -> eps`
30	`CondP ; $`	`; $`	`CondP -> eps`
31	`; $`	`; $`	`match ;`
32	`$`	`$`	`ACEPTAR`

Simulación paso a paso de la Pila: Documentación en el informe de la traza completa de la pila sintáctica (paso, pila, entrada, acción) para al menos una consulta válida

Windows:

```
None
g++ -std=c++20 -Iinclude -Iparser_ll1/include src/lexer.cpp src/token.cpp src/input.cpp parser_ll1/src/grammar.cpp
parser_ll1/src/parser.cpp parser_ll1/src/parse_table.cpp parser_ll1/src/trace.cpp parser_ll1/src/main.cpp -o parser_ll1.exe
```

crear la consulta:

```
None
"SELECT nombre, edad FROM alumnos WHERE edad >= 18 AND ciclo = 3;" | Out-File -Encoding ascii consulta.sql
```

Ejecutar y ver la traza:
.\parser_ll1.exe consulta.sql

```
None
.\parser_ll1.exe consulta.sql
.\parser_ll1.exe consulta.sql > traza.txt
```

Linux (Ubuntu):

./parser_ll1 ../parser_ll1/examples/parser_valid_complex.sql

```
luisenramos@ElAprendiz2005:~/Documentos/EDA/EDA-Lab04/build$ ./parser_ll1 ../parser_ll1/examples/parser_valid_complex.sql
Consulta:
SELECT nombre, edad, carrera
FROM estudiantes
WHERE edad >= 18 AND carrera = 'CC' OR promedio <> 10.5 AND codigo != 7;


Traza del parseo:
Paso  Pila (tope a la izq.)                        Entrada                                    Accion
-------------------------------------------------------------------------------------------------------------------
1     Query $                                      SELECT nombre , edad , carrera FROM estudiantes WHERE edad >= 18 AND carrera = 'CC' OR promedio <> 10.5 AND codigo != 7 ; $  Query -> SELECT
SelectList FROM id WhereOpt ;
2     SELECT SelectList FROM id WhereOpt ; $       SELECT nombre , edad , carrera FROM estudiantes WHERE edad >= 18 AND carrera = 'CC' OR promedio <> 10.5 AND codigo != 7 ; $  match SELECT
3     SelectList FROM id WhereOpt ; $              nombre , edad , carrera FROM estudiantes WHERE edad >= 18 AND carrera = 'CC' OR promedio <> 10.5 AND codigo != 7 ; $  SelectList -> id IdList
P
4     id IdListP FROM id WhereOpt ; $              nombre , edad , carrera FROM estudiantes WHERE edad >= 18 AND carrera = 'CC' OR promedio <> 10.5 AND codigo != 7 ; $  match id
5     IdListP FROM id WhereOpt ; $                 , edad , carrera FROM estudiantes WHERE edad >= 18 AND carrera = 'CC' OR promedio <> 10.5 AND codigo != 7 ; $  IdListP -> , id IdListP
6     , id IdListP FROM id WhereOpt ; $            , edad , carrera FROM estudiantes WHERE edad >= 18 AND carrera = 'CC' OR promedio <> 10.5 AND codigo != 7 ; $  match ,
7     id IdListP FROM id WhereOpt ; $              edad , carrera FROM estudiantes WHERE edad >= 18 AND carrera = 'CC' OR promedio <> 10.5 AND codigo != 7 ; $  match id
8     IdListP FROM id WhereOpt ; $                 , carrera FROM estudiantes WHERE edad >= 18 AND carrera = 'CC' OR promedio <> 10.5 AND codigo != 7 ; $  IdListP -> , id IdListP
9     , id IdListP FROM id WhereOpt ; $            , carrera FROM estudiantes WHERE edad >= 18 AND carrera = 'CC' OR promedio <> 10.5 AND codigo != 7 ; $  match ,
10    id IdListP FROM id WhereOpt ; $              carrera FROM estudiantes WHERE edad >= 18 AND carrera = 'CC' OR promedio <> 10.5 AND codigo != 7 ; $  match id
11    IdListP FROM id WhereOpt ; $                 FROM estudiantes WHERE edad >= 18 AND carrera = 'CC' OR promedio <> 10.5 AND codigo != 7 ; $  IdListP -> eps
12    FROM id WhereOpt ; $                         FROM estudiantes WHERE edad >= 18 AND carrera = 'CC' OR promedio <> 10.5 AND codigo != 7 ; $  match FROM
13    id WhereOpt ; $                              estudiantes WHERE edad >= 18 AND carrera = 'CC' OR promedio <> 10.5 AND codigo != 7 ; $  match id
14    WhereOpt ; $                                 WHERE edad >= 18 AND carrera = 'CC' OR promedio <> 10.5 AND codigo != 7 ; $  WhereOpt -> WHERE Cond
15    WHERE Cond ; $                               WHERE edad >= 18 AND carrera = 'CC' OR promedio <> 10.5 AND codigo != 7 ; $  match WHERE
16    Cond ; $                                     edad >= 18 AND carrera = 'CC' OR promedio <> 10.5 AND codigo != 7 ; $  Cond -> AndCond CondP
17    AndCond CondP ; $                            edad >= 18 AND carrera = 'CC' OR promedio <> 10.5 AND codigo != 7 ; $  AndCond -> Pred AndP
18    Pred AndP CondP ; $                          edad >= 18 AND carrera = 'CC' OR promedio <> 10.5 AND codigo != 7 ; $  Pred -> id RelOp Value
19    id RelOp Value AndP CondP ; $                edad >= 18 AND carrera = 'CC' OR promedio <> 10.5 AND codigo != 7 ; $  match id
20    RelOp Value AndP CondP ; $                   >= 18 AND carrera = 'CC' OR promedio <> 10.5 AND codigo != 7 ; $  RelOp -> >=
21    >= Value AndP CondP ; $                      >= 18 AND carrera = 'CC' OR promedio <> 10.5 AND codigo != 7 ; $  match >=
22    Value AndP CondP ; $                         18 AND carrera = 'CC' OR promedio <> 10.5 AND codigo != 7 ; $  Value -> int
23    int AndP CondP ; $                           18 AND carrera = 'CC' OR promedio <> 10.5 AND codigo != 7 ; $  match int
24    AndP CondP ; $                               AND carrera = 'CC' OR promedio <> 10.5 AND codigo != 7 ; $  AndP -> AND Pred AndP
25    AND Pred AndP CondP ; $                      AND carrera = 'CC' OR promedio <> 10.5 AND codigo != 7 ; $  match AND
26    Pred AndP CondP ; $                          carrera = 'CC' OR promedio <> 10.5 AND codigo != 7 ; $  Pred -> id RelOp Value
27    id RelOp Value AndP CondP ; $                carrera = 'CC' OR promedio <> 10.5 AND codigo != 7 ; $  match id
28    RelOp Value AndP CondP ; $                   = 'CC' OR promedio <> 10.5 AND codigo != 7 ; $  RelOp -> =
29    = Value AndP CondP ; $                       = 'CC' OR promedio <> 10.5 AND codigo != 7 ; $  match =
30    Value AndP CondP ; $                         'CC' OR promedio <> 10.5 AND codigo != 7 ; $  Value -> string
31    string AndP CondP ; $                        'CC' OR promedio <> 10.5 AND codigo != 7 ; $  match string
```

```
32    AndP CondP ; $                          OR promedio <> 10.5 AND codigo != 7 ; $   AndP -> eps
33    CondP ; $                               OR promedio <> 10.5 AND codigo != 7 ; $   CondP -> OR AndCond CondP
34    OR AndCond CondP ; $                    OR promedio <> 10.5 AND codigo != 7 ; $   match OR
35    AndCond CondP ; $                       promedio <> 10.5 AND codigo != 7 ; $      AndCond -> Pred AndP
36    Pred AndP CondP ; $                     promedio <> 10.5 AND codigo != 7 ; $      Pred -> id RelOp Value
37    id RelOp Value AndP CondP ; $           promedio <> 10.5 AND codigo != 7 ; $      match id
38    RelOp Value AndP CondP ; $              <> 10.5 AND codigo != 7 ; $               RelOp -> <>
39    <> Value AndP CondP ; $                 <> 10.5 AND codigo != 7 ; $               match <>
40    Value AndP CondP ; $                    10.5 AND codigo != 7 ; $                  Value -> float
41    float AndP CondP ; $                    10.5 AND codigo != 7 ; $                  match float
42    AndP CondP ; $                          AND codigo != 7 ; $                       AndP -> AND Pred AndP
43    AND Pred AndP CondP ; $                 AND codigo != 7 ; $                       match AND
44    Pred AndP CondP ; $                     codigo != 7 ; $                           Pred -> id RelOp Value
45    id RelOp Value AndP CondP ; $           codigo != 7 ; $                           match id
46    RelOp Value AndP CondP ; $              != 7 ; $                                  RelOp -> <>
47    <> Value AndP CondP ; $                 != 7 ; $                                  match <>
48    Value AndP CondP ; $                    7 ; $                                     Value -> int
49    int AndP CondP ; $                      7 ; $                                     match int
50    AndP CondP ; $                          ; $                                       AndP -> eps
51    CondP ; $                               ; $                                       CondP -> eps
52    ; $                                     ; $                                       match ;
53    $                                       $                                         ACEPTAR

RESULTADO: ACEPTADA (sintacticamente valida)
Producciones aplicadas: 1 3 4 4 5 6 8 11 14 20 21 12 14 15 23 13 9 11 14 16 22 12 14 16 21 13 10
```

10. Pruebas y resultados

Las pruebas se ejecutan con `./test_parser` (o `ctest`). Resultado actual: Las 64 pruebas del (lexer) + 36 (parser) + 41 (parser_tabla) también pasan, y las pruebas del parser se ejecutaron sin errores bajo AddressSanitizer y UndefinedBehaviorSanitizer.

Escenario	Título	Pruebas	Qué verifica
1	Gramatica: estructura y simbolos	3	Estructura: 24 producciones, símbolo inicial, sin recursión izquierda directa, texto de las producciones.
2	Conjuntos FIRST y FOLLOW (coinciden con docs/gramatica.md)	4	FIRST, FOLLOW, anulables y conjuntos de selección contra la tabla de referencia (25 celdas, sin conflictos).
3	Motor por pila: consultas validas simples	6	Consultas válidas simples: SELECT *, una y varias columnas, minúsculas, saltos de línea, WHERE simple.
4	Motor por pila: consultas validas complejas	5	Consultas válidas complejas: 7 operadores, 5 tipos de valor, AND/OR combinados, 500 columnas y 200 condiciones.
5	Motor por pila: comportamiento del algoritmo	3	Algoritmo: producciones aplicadas, observador (10 pasos, termina en ACEPTAR), pila final = $.
6	Motor por pila: reporte basico de error (todo o nada)	6	Errores básicos: posición, token encontrado, mensaje completo, fin de entrada, multilínea, texto tras “;”.
7	Motor por pila: robustez e integracion con el lexer	9	Robustez: entrada vacía, vector sin EOF o vacío, mapeo de tokens, decimales normalizados, palabras reservadas, cadenas con escape.

Pruebas :

```
None
g++ -std=c++20 -Iinclude src/lexer.cpp src/token.cpp src/input.cpp
tests/test_lexer.cpp -o test_lexer.exe

g++ -std=c++20 -Iinclude -Iparser_ll1/include src/lexer.cpp src/token.cpp
src/input.cpp parser_ll1/src/grammar.cpp parser_ll1/src/parser.cpp
parser_ll1/src/parse_table.cpp parser_ll1/src/trace.cpp
parser_ll1/tests/test_parser.cpp -o test_parser.exe

g++ -std=c++20 -Iinclude -Iparser_ll1/include src/lexer.cpp src/token.cpp
src/input.cpp parser_ll1/src/grammar.cpp parser_ll1/src/parser.cpp
parser_ll1/src/parse_table.cpp parser_ll1/src/trace.cpp
parser_ll1/tests/test_tabla_errores.cpp -o test_parser_tabla.exe

.\test_lexer.exe
.\test_parser.exe
.\test_parser_tabla.exe
```

lexer

```
PS C:\Users\HP SUPPORT\Downloads\EDA-Lab04-main2> .\test_lexer.exe
>>
Pruebas del analizador lexico SQL
================================

Escenario 1: Sentencia valida (SELECT, FROM, WHERE)
  [PASS] ejemplo del enunciado (tabla 1)
  [PASS] se conserva el orden y los lexemas
  [PASS] posiciones (linea, columna) en varias lineas
  [PASS] consulta con AND, OR, cadena y decimal

Escenario 2: Operadores compuestos
  [PASS] los seis operadores relacionales y '!='
  [PASS] >=, <=, <> y != son UN solo token con su lexema completo
  [PASS] '*' es el token STAR (SELECT *)
  [PASS] operadores sin espacios alrededor
  [PASS] operadores separados por espacio NO se unen
  [PASS] operadores consecutivos se dividen de forma maximal
  [PASS] '<' y '>' solos

Escenario 3: Literales (enteros, decimales y cadenas)
  [PASS] entero
  [PASS] decimal
  [PASS] entero negativo (el '-' va pegado al numero)
  [PASS] decimal negativo
  [PASS] .5 se normaliza a 0.5 (con advertencia)
  [PASS] 5. se normaliza a 5.0 (con advertencia)
  [PASS] -.5 se normaliza a -0.5
  [PASS] 5. antes de ';' no consume el punto y coma
  [PASS] a-3 se lee como identificador y entero negativo
  [PASS] cadena simple: el lexema no incluye las comillas
  [PASS] cadena vacia y cadena con espacios
  [PASS] palabras reservadas y operadores dentro de una cadena son texto
  [PASS] escape \' dentro de la cadena
  [PASS] escape \\ (barra invertida literal)
  [PASS] escape desconocido: advertencia, no error, se conserva el texto
  [PASS] caracteres no ASCII dentro de una cadena se aceptan
```

```
Escenario 4: Palabras reservadas vs identificadores
  [PASS] las cinco palabras reservadas
  [PASS] palabras reservadas insensibles a mayusculas/minusculas
  [PASS] el lexema conserva la escritura original
  [PASS] identificadores sensibles a mayusculas/minusculas
  [PASS] un identificador que CONTIENE una palabra reservada no es palabra reservada
  [PASS] identificadores validos: letras, digitos y guion bajo
  [PASS] una palabra reservada dentro de una sentencia no rompe el resto

Escenario 5: Espacios en blanco (espacios, tabulaciones, saltos de linea)
  [PASS] espacios multiples se ignoran
  [PASS] tabulaciones se ignoran y cuentan como una columna
  [PASS] saltos de linea incrementan la linea y reinician la columna
  [PASS] CRLF (\r\n) cuenta como un solo salto de linea
  [PASS] mezcla de espacios, tabs y saltos de linea en una consulta
  [PASS] los espacios separan tokens: 'ab' y 'a b' son distintos

Escenario 6: Errores lexicos
  [PASS] caracteres no admitidos (@ # $ % ^ & * ( ) ~ ?)
  [PASS] el error no se clasifica como token y el analisis continua
  [PASS] cadena sin cerrar (fin de entrada)
  [PASS] cadena con salto de linea real: error y la linea siguiente se analiza
  [PASS] la comilla escapada no cierra la cadena
  [PASS] '!' solo es error; '!=' es valido
  [PASS] '-' suelto es error (solo es valido pegado a un numero)
  [PASS] identificador que empieza por digito: un solo error
  [PASS] numero con mas de un punto: un solo error
  [PASS] caracter no ASCII fuera de cadena: un solo error por caracter
  [PASS] se acumulan todos los errores, en orden, con su posicion
  [PASS] los mensajes de error no estan vacios y todos son ERROR
  [PASS] las advertencias no cuentan como errores
  [PASS] hasErrors() refleja el estado del lexer
```

```
Escenario 7: Casos limite
  [PASS] entrada vacia: solo EOF en 1:1
  [PASS] solo espacios y saltos de linea: solo EOF con su posicion
  [PASS] sentencia incompleta: el lexer no valida la gramatica
  [PASS] token al final de la entrada, sin espacio ni salto de linea
  [PASS] operador de dos caracteres incompleto al final
  [PASS] un unico token
  [PASS] siempre termina en un unico EOF
  [PASS] nextToken() repetido tras el EOF sigue devolviendo EOF
  [PASS] nextToken() omite los errores y entrega el siguiente token valido
  [PASS] entrada larga (rendimiento y posiciones)

==============================
Pruebas ejecutadas: 64 | verificaciones: 321 | fallidas: 0
RESULTADO: TODAS LAS PRUEBAS PASARON
PS C:\Users\HP SUPPORT\Downloads\EDA-Lab04-main2>
```

parser

```
cpp parser_ll1/src/parser.cpp parser_ll1/src/parse_table.cpp parser_ll1/src/trace.cpp parser_ll1/tests/test_parser.cpp -o test_parser.exe
PS C:\Users\HP SUPPORT\Downloads\EDA-Lab04-main2> .\test_parser.exe

Escenario 1: Gramatica: estructura y simbolos
  [PASS] la gramatica tiene 24 producciones y simbolo inicial Query
  [PASS] sin recursion izquierda directa
  [PASS] producciones con el texto documentado

Escenario 2: Conjuntos FIRST y FOLLOW (coinciden con docs/gramatica.md)
  [PASS] FIRST de cada no terminal
  [PASS] no terminales anulables (derivan epsilon)
  [PASS] FOLLOW de cada no terminal
  [PASS] conjuntos de seleccion = celdas de la tabla de referencia (LL(1) sin conflictos)

Escenario 3: Motor por pila: consultas validas simples
  [PASS] SELECT * FROM t;
  [PASS] SELECT a FROM t;
  [PASS] varias columnas
  [PASS] palabras clave en minusculas y mezcla
  [PASS] saltos de linea y tabuladores
  [PASS] WHERE con una comparacion

Escenario 4: Motor por pila: consultas validas complejas
  [PASS] todos los operadores relacionales
  [PASS] todos los tipos de valor
  [PASS] AND, OR y combinaciones
  [PASS] proyeccion multiple + WHERE compuesto (archivo de ejemplo)
  [PASS] entrada larga: 500 columnas y 200 condiciones

Escenario 5: Motor por pila: comportamiento del algoritmo
  [PASS] producciones aplicadas en orden (derivacion por la izquierda)
  [PASS] el observador recibe un evento por paso (10 pasos) y el ultimo es ACEPTAR
  [PASS] la pila termina vacia: solo '$' en el fondo al aceptar

Escenario 6: Motor por pila: reporte basico de error (todo o nada)
  [PASS] se detiene en el primer error: linea, columna y token encontrado
  [PASS] mensaje completo con posicion, token y esperados
  [PASS] terminal esperado distinto del token (FROM antes de SELECT)
  [PASS] fin de entrada inesperado (falta ';') informa los tokens esperados
  [PASS] posicion correcta en entrada multilinea
  [PASS] tokens despues del ';' se rechazan

Escenario 7: Motor por pila: robustez e integracion con el lexer
  [PASS] entrada vacia se rechaza sin fallar
  [PASS] solo espacios, y un unico token
  [PASS] vector de tokens vacio (sin EOF) no se sale de rango
  [PASS] vector sin TOKEN_EOF final se trata como fin de entrada
  [PASS] correspondencia TokenType -> Terminal
  [PASS] el lexer no se modifica: != y <> son el mismo terminal
  [PASS] decimales normalizados por el lexer (.5 y 5.) son validos
  [PASS] palabras reservadas no pueden usarse como identificador
  [PASS] cadena con escapes cuenta como un solo valor

================================
Pruebas ejecutadas: 36 | verificaciones: 162 | fallidas: 0
RESULTADO: TODAS LAS PRUEBAS PASARON
```

```
.\test_parser_tabla.exe
```

```
PS C:\Users\HP SUPPORT\Downloads\EDA-Lab04-main2> .\test_parser_tabla.exe
>>

Escenario 1: Tabla M[A, a] construida desde FIRST/FOLLOW
  [PASS] la gramatica es LL(1): la tabla no tiene conflictos
  [PASS] la tabla tiene 25 celdas no vacias
  [PASS] celdas de docs/gramatica.md (seccion 7)
  [PASS] las celdas vacias devuelven -1
  [PASS] expected() lista los terminales de la fila
  [PASS] una gramatica ambigua SI produce conflictos (el detector funciona)
  [PASS] printParseTable imprime la matriz y 'Sin conflictos'

Escenario 2: Consultas validas con la tabla construida
  [PASS] simples
  [PASS] complejas

Escenario 3: Traza de la pila: paso, pila, entrada y accion
  [PASS] traza de SELECT a FROM t WHERE x = 1;
  [PASS] stackToString: tope a la izquierda
  [PASS] remainingInput: entrada restante termina en $
  [PASS] la traza de una consulta invalida termina en ERROR

Escenario 4: Errores: palabras clave fuera de orden
  [PASS] FROM antes de SELECT
  [PASS] WHERE antes de FROM
  [PASS] sin SELECT

Escenario 5: Errores: omision de tokens obligatorios
  [PASS] falta la tabla despues de FROM
  [PASS] falta la tabla y no hay mas entrada
  [PASS] falta ';' al final
  [PASS] falta ';' despues de WHERE
  [PASS] falta la lista de columnas
  [PASS] falta FROM
  [PASS] coma sobrante antes de FROM
```

```
  Escenario 6: Errores: expresiones mal formadas
    [PASS] falta el valor de la comparacion
    [PASS] falta el operador relacional
    [PASS] falta la columna izquierda
    [PASS] WHERE sin condicion
    [PASS] AND sin segunda condicion
    [PASS] dos operadores logicos seguidos
    [PASS] operador logico al inicio
    [PASS] dos operadores relacionales seguidos
    [PASS] comparacion con '*' como valor
    [PASS] tokens sobrantes despues de ';'

  Escenario 7: Errores lexicos y robustez (nunca debe haber crash)
    [PASS] el lexer reporta un caracter invalido
    [PASS] cadena sin cerrar: el lexer marca error
    [PASS] el parser no falla con los tokens que deja el lexer tras un error
    [PASS] entradas raras se rechazan sin fallar
    [PASS] el mensaje de error trae posicion, token encontrado y esperados

  ==================================
  Pruebas ejecutadas: 40 | verificaciones: 150 | fallidas: 0
  RESULTADO: TODAS LAS PRUEBAS PASARON
  PS C:\Users\HP SUPPORT\Downloads\EDA-Lab04-main2>
```

Anexo. Estructura del repositorio y compilación

Git HUB

None

https://github.com/LuisEn2005/EDA-Lab04.git
LuisEn2005/EDA-Lab04 at feature/analizador_sintactico_parte_completa

```
EDA-Lab04/
├── include/                  Práctica 4: lexer (token.h, lexer.h, input.h)
├── src/                      lexer.cpp, token.cpp, input.cpp, main.cpp
├── tests/                    test_lexer.cpp (64 pruebas)
├── examples/                 Ejemplos del lexer
├── parser_ll1/
parser_ll1/
├── docs/
│   ├── gramatica.md          Gramática, FIRST/FOLLOW y tabla M[A,a]
│   └── traza_ejemplo.txt     Traza de ejemplo de la pila
├── examples/
│   ├── parser_valid_simple.sql          Consulta válida simple
│   ├── parser_valid_complex.sql         Consulta válida compleja
│   ├── parser_invalid.sql               Palabras clave en desorden
│   ├── parser_invalid_sin_tabla.sql     Falta el nombre de la tabla
│   ├── parser_invalid_sin_puntoycoma.sql Falta el delimitador ;
│   ├── parser_invalid_coma.sql          Coma sobrante
│   ├── parser_invalid_expresion.sql     Expresión mal formada
│   └── parser_invalid_lexico.sql        Error léxico (@)
├── include/
│   ├── grammar.h             Símbolos, producciones, FIRST/FOLLOW
│   ├── parse_table.h         Tabla de parseo M[A,a]
│   ├── parser.h              Motor LL(1) con pila
│   └── trace.h               Traza paso a paso
├── src/
│   ├── grammar.cpp           24 producciones y cálculo de FIRST/FOLLOW
│   ├── parse_table.cpp       Construcción de la tabla y detección de conflictos
│   ├── parser.cpp            Algoritmo predictivo no recursivo y errores
│   ├── trace.cpp             Impresión de la traza por consola
│   └── main.cpp              Ejecutable parser_ll1
└── tests/
    ├── test_parser.cpp       36 pruebas: gramática, FIRST/FOLLOW, motor
    └── test_tabla_errores.cpp  40 pruebas: tabla, traza, consultas inválidas
            Práctica 5 (estructura de arriba)
├── CMakeLists.txt
├── README.md
└── .gitignore
```